

returns 1 since the first and second Fibonacci numbers are always 1. Any other case is a recursive case, so the value that is returned is the sum of `fibonacci(nthNumber - 1)` and `fibonacci(nthNumber - 2)`. As long as the original `nthNumber` argument is an integer greater than 0, these recursive calls will eventually reach the base case and stop making more recursive calls.

Remember how the recursive factorial example had a “before the recursive call” and “after the recursive call” part? Because the recursive Fibonacci algorithm makes two recursive calls in its recursive case, you should keep in mind that it has three parts: “before the first recursive call,” “after the first recursive call but before the second recursive call,” and “after the second recursive call.” But the same principles apply. And don’t think that because a base case is reached, no more code remains to run after either recursive call. The recursive algorithm is finished only after the original function call has returned.

You might ask, “Isn’t the iterative Fibonacci solution simpler than the recursive Fibonacci solution?” The answer is “Yes.” Even worse, the recursive solution has a critical inefficiency that is explained in the next section.

## ***Why the Recursive Fibonacci Algorithm Is Terrible***

Like the recursive factorial algorithm, the recursive Fibonacci algorithm also suffers from a critical weakness: it repeats the same calculations over and over. [Figure 2-3](#) shows how calling `fibonacci(6)`, marked in the tree diagram as `fib(6)` for brevity, calls `fibonacci(5)` and `fibonacci(4)`.